TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH

BÁO CÁO CUỐI KỲ

Môn học: Phân tích dữ liệu lớn

Phân tích dữ liệu cảm biến và dự đoán tuổi thọ hữu ích còn lại của động cơ

NASA C-MAPSS FD001

Đề tài 8: Phân tích dữ liệu IoT và bảo trì dự đoán

Học viên thực hiện:	Phan Nguyễn Mai Phương
Mã số học viên:	25002711
Ngành:	Khoa học máy tính

TP. Hồ Chí Minh, tháng 9 năm 2026

Tóm tắt

Báo cáo trình bày quy trình phân tích dữ liệu cảm biến trên NASA C-MAPSS FD001, từ kiểm tra chất lượng dữ liệu đến xây dựng đặc trưng rolling window và suy luận thống kê theo động cơ, sau đó huấn luyện mô hình RUL bằng đánh giá grouped cross-validation. Kết quả hiện có gồm hồ sơ dữ liệu train/test, pipeline ETL Silver/Gold với đối chiếu Pandas–PySpark, 27.927 rolling windows dài 30 cycles, phân tích chênh lệch giữa 30 cycles đầu/cuối của 100 động cơ train và đánh giá mô hình trên 100 động cơ test.

Các kết quả được diễn giải trong phạm vi dữ liệu mô phỏng FD001. Báo cáo không đưa ra kết quả ARIMA hoặc dashboard vì những kết quả đó chưa thuộc tập bằng chứng hiện có; do đó không suy ra lịch bảo trì tối ưu từ các phân tích mô tả, suy luận sensor hoặc metric mô hình.

Mục lục

Tóm tắt i

Bảng thuật ngữ và chữ viết tắt iv

1 Bài toán, mục tiêu và phạm vi 1
1.1 Bài toán nghiên cứu . . . 1
1.2 Câu hỏi và giới hạn nghiên cứu . . . 1

2 Dữ liệu và kiểm soát chất lượng 2
2.1 Nguồn dữ liệu và lý do lựa chọn . . . 2
2.2 Từ điển dữ liệu và ý nghĩa cột . . . 2
2.3 Hồ sơ cột trên tập train . . . 3
2.4 Đánh giá phân bố và cách xử lý dữ liệu . . . 5
2.5 Các kiểm tra đã triển khai và lý do . . . 5
2.6 Tính toàn vẹn và giới hạn của validation . . . 6

3 Cơ sở lý thuyết và nghiên cứu liên quan 7
3.1 Prognostics và RUL . . . 7
3.2 Biểu diễn chuỗi bằng trailing window . . . 7
3.3 Paired analysis, bootstrap và nhiều kiểm định . . . 8
3.4 Spark, lưu trữ và giới hạn benchmark . . . 8

4 Thiết kế phương pháp và lý do lựa chọn 9
4.1 Luồng xử lý . . . 9
4.2 EDA theo đơn vị động cơ . . . 10
4.3 Đặc trưng từ trailing window . . . 10
4.4 Mô hình dự đoán RUL . . . 10
4.5 Suy luận thống kê . . . 11

5 Protocol suy luận thống kê 12
5.1 Mục tiêu và phạm vi . . . 12
5.2 Định nghĩa cặp đầu/cuối và ví dụ . . . 12
5.3 Đại lượng cần báo cáo và điều kiện diễn giải . . . 13
5.4 Cấu hình suy luận . . . 14

6 Kết quả và diễn giải 16
6.1 Kiểm tra dữ liệu . . . 16
6.2 ETL Silver/Gold và benchmark . . . 16
6.3 Rolling features . . . 16
6.4 Mô hình RUL và đánh giá cuối . . . 17
6.5 Tuổi thọ động cơ trên train . . . 19
6.6 Sensor hằng và correlation theo engine . . . 19
6.7 Suy luận đầu/cuối vòng đời trên train . . . 20

7 Khó khăn, bài học và giới hạn **23**

Tài liệu tham khảo **24**

Bảng thuật ngữ và chữ viết tắt

Thuật ngữ	Ý nghĩa trong báo cáo
Remaining Useful Life (RUL)	Tuổi thọ hữu ích còn lại, tính bằng số cycle trước khi động cơ hỏng.
NASA C-MAPSS / FD001	Bộ dữ liệu mô phỏng chuỗi vận hành turbofan; FD001 có một điều kiện vận hành và một dạng suy giảm HPC.
Engine / `engine_id`	Đơn vị động cơ và ranh giới của các phép ghép cặp, window và tổng hợp.
Cycle	Bước vận hành rời rạc trong lịch sử một engine; không quy đổi thành giờ hoặc ngày.
Sensor	Kênh đo tín hiệu; FD001 có 21 sensor trong mỗi observation.
Train / test	Train có lịch sử đến khi hỏng để tạo nhãn RUL; test bị cắt trước khi hỏng và có nhãn đánh giá tách riêng.
Data contract	Tập điều kiện hợp lệ của schema, kiểu số, khóa duy nhất, split và cycle continuity.
Exploratory Data Analysis (EDA)	Phân tích khám phá phân phối, biến thiên và tín hiệu trước khi xây dựng feature.
Extract, Transform, Load (ETL)	Quy trình đọc, biến đổi và ghi dữ liệu qua các lớp lưu trữ.
Bronze / Silver / Gold	Bronze giữ raw; Silver lưu observations và train targets đã kiểm tra; Gold lưu engine summary và rolling features.
Trailing window	Đoạn quan sát liên tiếp gần nhất của một engine tại cycle đang xét; báo cáo dùng 30 cycles.
Feature	Biến được tạo từ observations; mỗi sensor có last, mean, sample standard deviation, min, max và slope.
Paired analysis	Ghép hai phép đo của cùng một engine; protocol dùng 30 cycles đầu và 30 cycles cuối.
Bootstrap	Lấy mẫu lại có hoàn lại để ước lượng độ bất định; engine là đơn vị lấy mẫu.
Wilcoxon signed-rank test	Kiểm định phi tham số cho dữ liệu ghép cặp dựa trên thứ hạng và dấu chênh lệch.
Benjamini–Hochberg FDR	Hiệu chỉnh nhiều p-value trong họ 21 sensor để kiểm soát False Discovery Rate.
Parquet / SQLite	Parquet lưu bảng phân tích; SQLite cung cấp lớp truy vấn local cho engine summary.
Spark / PySpark	Framework xử lý dữ liệu phân tán và API Python dùng để đối chiếu ETL với Pandas.

Danh mục hình

1 Luồng xử lý dữ liệu đã thực hiện . . . 9
2 Protocol thống kê đầu/cuối vòng đời . . . 13
3 RUL thật và RUL dự đoán trên test . . . 18
4 Sai số dự đoán theo RUL thật . . . 18
5 Phân phối tuổi thọ động cơ trên train . . . 19
6 Phân phối correlation Spearman sensor–RUL theo động cơ . . . 20
7 Phân phối chênh lệch sensor đầu/cuối theo engine . . . 21
8 Chênh lệch cặp trung bình và khoảng tin cậy bootstrap . . . 22

Danh mục bảng

2 Quy mô FD001 đã ghi nhận trong báo cáo validation. . . . 2
3 Data dictionary của FD001 trong repository. . . . 2
4 Hồ sơ nhóm cột trên train FD001. . . . 4
5 Profile 21 sensor trên train FD001. . . . 4
6 Đặc điểm phân bố dữ liệu và cách xử lý. . . . 5
7 Benchmark Pandas tạo engine summary . . . 16
8 Rolling features theo split . . . 16
9 Cross-validation mô hình RUL . . . 17
10 Metric cuối trên NASA test FD001 . . . 17
11 Thống kê tuổi thọ của 100 động cơ train . . . 19
12 Correlation Spearman theo engine: các dòng minh họa . . . 19
13 Ví dụ kết quả suy luận sensor đầu/cuối . . . 20

1 Bài toán, mục tiêu và phạm vi

1.1 Bài toán nghiên cứu

Đề tài phân tích chuỗi cảm biến động cơ nhằm ước lượng số cycle còn lại trước khi hỏng. Đầu vào tại cycle t là các quan sát đã có đến t của một động cơ; đầu ra mục tiêu là RUL tại cycle đó. Phạm vi kết quả trong báo cáo gồm kiểm tra dữ liệu, EDA, ETL, rolling features và suy luận thống kê đầu/cuối vòng đời.

Đối với train, đặt T_i là cycle cuối của động cơ i trong lịch sử đầy đủ đến khi hỏng. Nhãn RUL được xác định bởi:

$$y_{i,t} = T_i - t, \qquad 1 \leq t \leq T_i. \tag{1}$$

Trong đó:

+ i: engine đang được xét; đây là đơn vị độc lập trong split train.
+ t: cycle đang xét của engine i, tính theo số cycle vận hành rời rạc.
+ T_i: cycle cuối của engine i trong tập train, cũng tính bằng cycle.
+ $y_{i,t}$: RUL của engine i tại cycle t, tính bằng cycle.

Công thức 1 được dùng vì train FD001 là run-to-failure history: chuỗi train kết thúc tại thời điểm hỏng, còn mục tiêu của bộ dữ liệu là dự đoán số operational cycles còn lại trước khi hỏng [6, 9]. Đây là định nghĩa nhãn từ dữ liệu, không phải công thức dự đoán do mô hình học ra. Độ tin cậy của nhãn train phụ thuộc vào giả định của nguồn FD001 rằng train đã đi đến failure; vì vậy validator chỉ áp dụng công thức này cho train và không áp dụng cho test.

Ví dụ minh họa: một động cơ kết thúc ở cycle 200 có RUL bằng 30 cycles tại cycle 170 và bằng 0 tại cycle 200. Đây là ví dụ tính nhãn, không phải kết quả dự đoán.

Lịch sử test bị cắt trước khi hỏng nên không được lấy cycle cuối quan sát của test làm T_i. Mục tiêu đánh giá cuối là một dự đoán tại cycle cuối quan sát cho mỗi test engine. Bộ nhãn chính thức được tách khỏi quá trình lựa chọn mô hình.

1.2 Câu hỏi và giới hạn nghiên cứu

Các câu hỏi được trả lời gồm: dữ liệu có đáp ứng contract không; cảm biến biến đổi thế nào theo vòng đời; ETL Pandas và PySpark có tạo kết quả tương đương không; và trailing window 30 cycles tạo ra tập đặc trưng có cấu trúc nào.

FD001 không có chi phí bảo trì, downtime hoặc lịch can thiệp. Do đó, RUL không tự động xác định thời điểm bảo trì tối ưu. Cycle cũng không được đổi thành giờ hoặc ngày vì dữ liệu không cung cấp thông tin quy đổi.

2 Dữ liệu và kiểm soát chất lượng

2.1 Nguồn dữ liệu và lý do lựa chọn

NASA C-MAPSS mô phỏng các chuỗi vận hành đến khi hỏng của động cơ turbofan. Nguồn dữ liệu được ghi nhận tại NASA Open Data [6]; tài liệu nền tảng đi kèm bộ dữ liệu là công trình của Saxena và cộng sự [9]. Mô tả đi kèm file raw xác định FD001 có một điều kiện vận hành và một dạng suy giảm High-Pressure Compressor (HPC, máy nén áp suất cao).

FD001 được chọn vì có lịch sử train đầy đủ đến khi hỏng, test bị cắt và nhãn đánh giá tách riêng, phù hợp với cách xây dựng bài toán RUL. Chỉ sử dụng một tập con giúp kiểm soát phạm vi thực nghiệm và làm rõ từng bước xử lý. Đổi lại, bằng chứng chỉ áp dụng trong điều kiện mô phỏng đang xét; không đủ để khẳng định khả năng khái quát trên động cơ thực tế hoặc điều kiện vận hành khác.

Bảng 2: Quy mô FD001 đã ghi nhận trong báo cáo validation.

Thành phần	Số dòng	Động cơ	Ý nghĩa
Train	20.631	100	Lịch sử đến khi hỏng
Test	13.096	100	Lịch sử bị cắt trước khi hỏng
Test labels	100	100	RUL tại cycle cuối quan sát

Mỗi dòng raw có 26 trường: engine ID, cycle, 3 operational settings và 21 sensors. Reader bổ sung cột `split`, tạo bảng observations có 27 cột. Khóa là bộ ba `(split, engine_id, cycle)`: train engine 1 và test engine 1 không phải cùng một thực thể. Nhãn test chỉ được kiểm tra cấu trúc riêng trong báo cáo integrity; việc này không phải đánh giá mô hình.

2.2 Từ điển dữ liệu và ý nghĩa cột

Dữ liệu train/test gốc không có header; tên cột trong báo cáo được đặt theo contract của repository. Bảng 3 mô tả vai trò từng nhóm cột và cách sử dụng trong phân tích. Nguồn NASA chỉ mô tả các trường sensor là "sensor measurement", không cung cấp tên vật lý, đơn vị đo hoặc mapping chi tiết cho từng sensor. Vì vậy, báo cáo giữ tên `sensor_1` đến `sensor_21`; không tự diễn giải một sensor cụ thể là nhiệt độ, áp suất hoặc tốc độ nếu chưa có nguồn xác nhận.

Bảng 3: Data dictionary của FD001 trong repository.

Cột hoặc nhóm cột	Ý nghĩa	Kiểu/miền đã kiểm tra	Vai trò trong báo cáo
`split`	Cột do reader bổ sung để phân biệt nguồn dữ liệu.	Chuỗi `train` hoặc `test`.	Giữ ranh giới train/test trong khóa dữ liệu và validation; không có trong raw text gốc.

Cột hoặc nhóm cột	Ý nghĩa	Kiểu/miền đã kiểm tra	Vai trò trong báo cáo
`engine_id`	Mã chuỗi động cơ trong từng split.	Số nguyên dương; FD001 có engine ID 1–100 ở cả train và test.	Đơn vị độc lập cho tuổi thọ, ghép cặp đầu/cuối, GroupKFold và đánh giá theo engine.
`cycle`	Thứ tự vận hành rời rạc của một engine.	Số nguyên dương, bắt đầu từ 1 và liên tục trong từng engine.	Trục thời gian rời rạc; dùng để tạo RUL train, window 30 cycles và phân tích đầu/cuối vòng đời.
`setting_1`–`setting_3`	Ba operational settings ảnh hưởng đến điều kiện vận hành.	`float64`; trên train, `setting_3` hằng ở 100.	Biến bối cảnh vận hành; không thuộc họ 21 sensor trong kiểm định M2.
`sensor_1`–`sensor_21`	Hai mươi mốt phép đo sensor trong mỗi cycle.	`float64` hữu hạn; không có missing trong train.	Nguồn tín hiệu chính cho EDA, paired analysis và feature engineering.
`rul` trong train targets	Nhãn RUL được suy ra từ lịch sử train đầy đủ đến khi hỏng.	Số nguyên không âm; trên train hiện có miền 0–361 cycles.	Biến mục tiêu cho supervised regression; không phải cột raw.
`RUL_FD001.txt`	Nhãn RUL chính thức tại cycle cuối quan sát của mỗi engine test.	100 số nguyên không âm, ánh xạ dòng i sang test engine i.	Chỉ được ghép sau khi đã tạo 100 dự đoán test cuối cùng; không dùng trong EDA hoặc model selection.

Nhãn kết cục của bài toán là RUL, tức số cycle còn lại trước khi engine hỏng. Trong train, nhãn được suy ra từ cycle cuối của từng chuỗi; trong test, nhãn chỉ có tại cycle cuối quan sát và nằm trong file riêng `RUL_FD001.txt`. Do đó, mỗi dòng train có thể có một nhãn RUL, còn mỗi engine test chỉ có một nhãn đánh giá cuối. Báo cáo không chuyển RUL thành nhãn phân loại hỏng/không hỏng vì nguồn dữ liệu và protocol hiện tại đánh giá sai số theo số cycle.

Ví dụ đọc một dòng raw: dòng đầu tiên của `train_FD001.txt` có `engine_id=1`, `cycle=1`, ba settings lần lượt là $-0{,}0007$, $-0{,}0004$, $100{,}0$, sau đó là 21 giá trị sensor. Engine 1 trong train kết thúc ở cycle 192, nên sau khi tạo nhãn, dòng này có `rul=191`. Đây là nhãn hồi cứu của train, không phải dự đoán của mô hình.

2.3 Hồ sơ cột trên tập train

Bảng 4 tóm tắt các nhóm cột sau khi ghép RUL train. Các thống kê dùng để mô tả cấu trúc dữ liệu trước khi xây feature, không phải bằng chứng về hiệu quả dự đoán.

Bảng 4: Hồ sơ nhóm cột trên train FD001.

Nhóm cột	Số cột	Missing	Nhận xét chính
Khóa `engine_id`, `cycle`	2	0	20.631 quan sát, 100 engine; không có khóa trùng, cycle liên tục trong từng engine.
Operational settings	3	0	`setting_1` có 158 giá trị khác nhau, `setting_2` có 13 giá trị, `setting_3` hằng bằng 100.
Sensors	21	0	Sáu sensor hằng toàn train; `sensor_6` chỉ có hai giá trị và hằng trong 38 engine.
Nhãn `rul` train	1	0	Miền 0–361 cycles; trung vị theo observation là 103 cycles.

Bảng 5: Profile 21 sensor trên train FD001.

Sensor	Số giá trị khác nhau	Min–max	Ghi chú
`sensor_1`	1	518,67–518,67	Hằng toàn train.
`sensor_2`	310	641,21–644,53	Biến thiên trên train.
`sensor_3`	3.012	1.571,04–1.616,91	Biến thiên trên train.
`sensor_4`	4.051	1.382,25–1.441,49	Biến thiên trên train; có chênh lệch đầu/cuối lớn trong M2.3a.
`sensor_5`	1	14,62–14,62	Hằng toàn train.
`sensor_6`	2	21,60–21,61	Biến thiên toàn train nhưng hằng trong 38/100 engine.
`sensor_7`	513	549,85–556,06	Biến thiên trên train.
`sensor_8`	53	2.387,90–2.388,56	Biến thiên thấp, nhưng không hằng.
`sensor_9`	6.403	9.021,73–9.244,59	Biến thiên mạnh; phân phối theo engine không đồng nhất hoàn toàn.
`sensor_10`	1	1,30–1,30	Hằng toàn train.
`sensor_11`	159	46,85–48,53	Biến thiên; Spearman median với RUL âm trong EDA.
`sensor_12`	427	518,69–523,38	Biến thiên; Spearman median với RUL dương trong EDA.
`sensor_13`	56	2.387,88–2.388,56	Biến thiên thấp, nhưng không hằng.
`sensor_14`	6.078	8.099,94–8.293,72	Biến thiên mạnh theo giá trị gốc.
`sensor_15`	1.918	8,3249–8,5848	Biến thiên trên train.
`sensor_16`	1	0,03–0,03	Hằng toàn train.
`sensor_17`	13	388–400	Biến thiên rời rạc trên train.
`sensor_18`	1	2.388–2.388	Hằng toàn train.
`sensor_19`	1	100–100	Hằng toàn train.
`sensor_20`	120	38,14–39,43	Biến thiên trên train.
`sensor_21`	4.745	22,8942–23,6184	Biến thiên trên train.

Các cột hằng toàn train không có correlation Spearman xác định với RUL và không thể tạo chênh lệch đầu/cuối khác 0 trong paired analysis. Báo cáo vẫn giữ chúng trong bảng thống kê để thể hiện đầy đủ phạm vi 21 sensor.

2.4 Đánh giá phân bố và cách xử lý dữ liệu

Dữ liệu FD001 tương đối sạch về mặt schema: không có missing trong các cột đã đọc, khóa `(split, engine_id, cycle)` không trùng và cycle liên tục trong từng engine. Tuy nhiên, dữ liệu không hoàn toàn thuận lợi cho mọi phân tích: độ dài chuỗi giữa các engine khác nhau, sáu sensor hằng trên toàn train, một số sensor có rất ít mức giá trị, và tài liệu gốc không cung cấp đơn vị vật lý cho từng sensor. Bảng 6 tóm tắt các điểm này và cách xử lý trong báo cáo.

Bảng 6: Đặc điểm phân bố dữ liệu và cách xử lý.

Đặc điểm	Ý nghĩa	Cách xử lý trong báo cáo
Không có missing và khóa hợp lệ	Dữ liệu đủ sạch để tạo EDA, ETL và feature artifact mà không cần impute.	Không tự điền dữ liệu; validator dừng khi schema, khóa hoặc cycle continuity sai.
Độ dài chuỗi khác nhau giữa engine	Engine sống lâu tạo nhiều observations và nhiều rolling windows hơn.	Báo cáo tách thống kê theo observation và theo engine; cross-validation và metric chính dùng engine làm đơn vị.
Sensor hằng hoặc ít biến thiên	Một số correlation hoặc kiểm định không xác định; sensor hằng không tạo chênh lệch đầu/cuối.	Giữ đủ 21 sensor trong bảng, gắn trạng thái không xác định hoặc all-zero; không xóa sensor chỉ để làm kết quả đẹp hơn.
Không có đơn vị vật lý cho sensor	Không thể diễn giải `sensor_k` là nhiệt độ, áp suất hoặc tốc độ cụ thể.	Dùng tên ẩn danh `sensor_1`– `sensor_21`; mọi chênh lệch được hiểu theo đơn vị gốc của sensor đó.
Test labels tách riêng	Nếu đọc nhãn test trước khi chọn mô hình sẽ gây data leakage.	Chỉ ghép `RUL_FD001.txt` sau khi đã ghi final predictions.

2.5 Các kiểm tra đã triển khai và lý do

Reader và validator thực hiện kiểm tra số trường, schema, kiểu số hữu hạn, bảng không rỗng, split hợp lệ, khóa duy nhất và cycle liên tục trong từng động cơ. Profile FD001 đối chiếu thêm số dòng, ID động cơ và cycle bắt đầu. Với train targets, validator kiểm tra đủ khóa và đối chiếu Công thức 1.

Các kiểm tra được đặt trước EDA vì dữ liệu sai có thể tạo ra thống kê hợp lệ về mặt tính toán nhưng sai về ý nghĩa. Chẳng hạn, chuỗi cycle $1, 2, 4$ không được coi là ba bước liên tiếp khi xây window. Hai dòng cùng khóa cũng không được giữ lại như hai phép đo độc lập. Khi

phát hiện lỗi, package báo `ContractError`; không tự điền dữ liệu, xóa dòng hoặc chỉnh raw để làm validation đạt.

2.6 Tính toàn vẹn và giới hạn của validation

Báo cáo JSON lưu SHA-256 của input để nhận biết phiên bản dữ liệu đã đọc. Dấu vân tay này chỉ hỗ trợ đối chiếu; tự tính checksum không chứng minh nguồn gốc file là chính xác. Việc kiểm tra row count và cycle continuity cũng không đủ phát hiện mọi thay đổi giá trị sensor.

Đặc biệt, một chuỗi train bị cắt đuôi nhưng vẫn liên tục có thể tạo ra RUL sai nếu bị hiểu nhầm là lịch sử đầy đủ. Vì vậy, raw được giữ bất biến, nguồn dữ liệu được ghi nhận và test labels được tách khỏi các phân tích train.

3 Cơ sở lý thuyết và nghiên cứu liên quan

3.1 Prognostics và RUL

Prognostics nghiên cứu diễn biến suy giảm và thời gian còn lại đến trạng thái hỏng. Với C-MAPSS FD001, train chứa chuỗi đến khi hỏng để tạo nhãn RUL, còn test chỉ cung cấp phần lịch sử trước thời điểm hỏng. Công trình của Saxena và cộng sự mô tả mô phỏng suy giảm và bài toán đánh giá prognostics [9]; tổng quan của Ramasso và Saxena làm rõ sự khác biệt giữa các tập con và protocol đánh giá [7].

Một dự đoán tại cycle cuối của mỗi test engine chỉ đánh giá chất lượng tại thời điểm cắt, không đại diện cho toàn bộ trajectory [8]. FD001 không có chi phí can thiệp hoặc downtime, nên RUL không trực tiếp xác định lịch bảo trì tối ưu.

3.2 Biểu diễn chuỗi bằng trailing window

Tại cycle t, một mẫu chỉ chứa các quan sát hiện có của cùng engine:

$$W_{i,t} = [\mathbf{x}_{i,t-w+1}, \ldots, \mathbf{x}_{i,t}]^{\mathsf{T}} \in \mathbb{R}^{w \times p}, \qquad t \geq w. \tag{2}$$

Trong đó:

+ i: engine đang được xét.
+ t: cycle cuối của window, tính bằng cycle vận hành.
+ w: số cycle trong window, với cấu hình hiện tại $w = 30$.
+ p: số sensor, với cấu hình hiện tại $p = 21$.
+ $\mathbf{x}_{i,t}$: vector sensor của engine i tại cycle t; mỗi phần tử giữ đơn vị gốc ẩn danh của sensor tương ứng.
+ $W_{i,t}$: ma trận đầu vào có w hàng và p cột, tức 30×21 trong cấu hình hiện tại.

Công thức 2 là định nghĩa biểu diễn do báo cáo triển khai, không phải một công thức dự đoán RUL do một tác giả bên ngoài đề xuất. Việc dùng window 30 cycles có đối chiếu với một cấu hình window RUL đã được sử dụng trong nghiên cứu trước, nhưng độ dài 30 không được coi là mặc định phổ quát [5]. Cách viết này được dùng để bảo đảm tại cycle t chỉ có dữ liệu quá khứ và hiện tại đi vào feature, nhờ đó tránh data leakage từ cycle tương lai. Độ tin cậy của biểu diễn được kiểm tra bằng validator rolling-window: mỗi window phải đủ 30 cycles, cùng engine và không nối sang engine khác. Ví dụ, mẫu tại cycle 170 dùng cycles 141–170 và không dùng cycle 171.

Mỗi sensor trong window được tóm tắt bằng giá trị cuối, mean, sample standard deviation, min, max và linear slope. Với dãy $z_1, \ldots, z_w$ của một sensor, slope được tính bởi

$$b = \frac{\sum_{k=1}^{w}(k - \bar{k})(z_k - \bar{z})}{\sum_{k=1}^{w}(k - \bar{k})^2}. \tag{3}$$

Trong đó:

+ k: vị trí cycle trong window, chạy từ 1 đến w.
+ w: độ dài window, hiện bằng 30 cycles.
+ z_k: giá trị sensor tại vị trí k, giữ đơn vị gốc của sensor đó.
+ $\bar{k}$: mean của các vị trí cycle trong window.
+ $\bar{z}$: mean của các giá trị sensor trong window.
+ b: mức thay đổi tuyến tính ước lượng trên mỗi cycle, có đơn vị "đơn vị sensor trên một cycle".

Slope trong Công thức 3 là hệ số hồi quy tuyến tính một biến theo ordinary least squares (OLS, bình phương tối thiểu thông thường) của giá trị sensor theo vị trí trong window [4]. Báo cáo dùng slope vì giá trị cuối và mean chưa mô tả hướng tăng/giảm bên trong 30 cycles gần nhất. Công thức này đáng tin như một tóm tắt xu hướng cục bộ có thể tái lập, nhưng không tự chứng minh utility dự đoán; utility chỉ được kiểm tra gián tiếp qua cross-validation của mô hình.

3.3 Paired analysis, bootstrap và nhiều kiểm định

Paired analysis ghép hai phép đo của cùng một engine để phân tích thay đổi trong vòng đời. Mean paired difference mô tả độ lớn thay đổi theo đơn vị gốc. Wilcoxon signed-rank dùng thứ hạng và dấu của chênh lệch; với giả định phân phối chênh lệch đối xứng quanh trung vị, kiểm định xem trung vị chênh lệch có bằng 0 hay không [13, 12]. Đây không phải kiểm định trực tiếp cho mean paired difference.

Bootstrap lấy mẫu lại engine có hoàn lại và giữ cặp đầu/cuối cùng nhau để ước lượng độ bất định [3, 10]. Với 21 sensor, Benjamini–Hochberg điều chỉnh p-value nhằm kiểm soát False Discovery Rate (FDR, tỷ lệ phát hiện sai kỳ vọng) trong họ kiểm định đã xác định [2, 11]. Khoảng tin cậy riêng từng sensor không trở thành simultaneous interval chỉ vì p-value đã được hiệu chỉnh.

3.4 Spark, lưu trữ và giới hạn benchmark

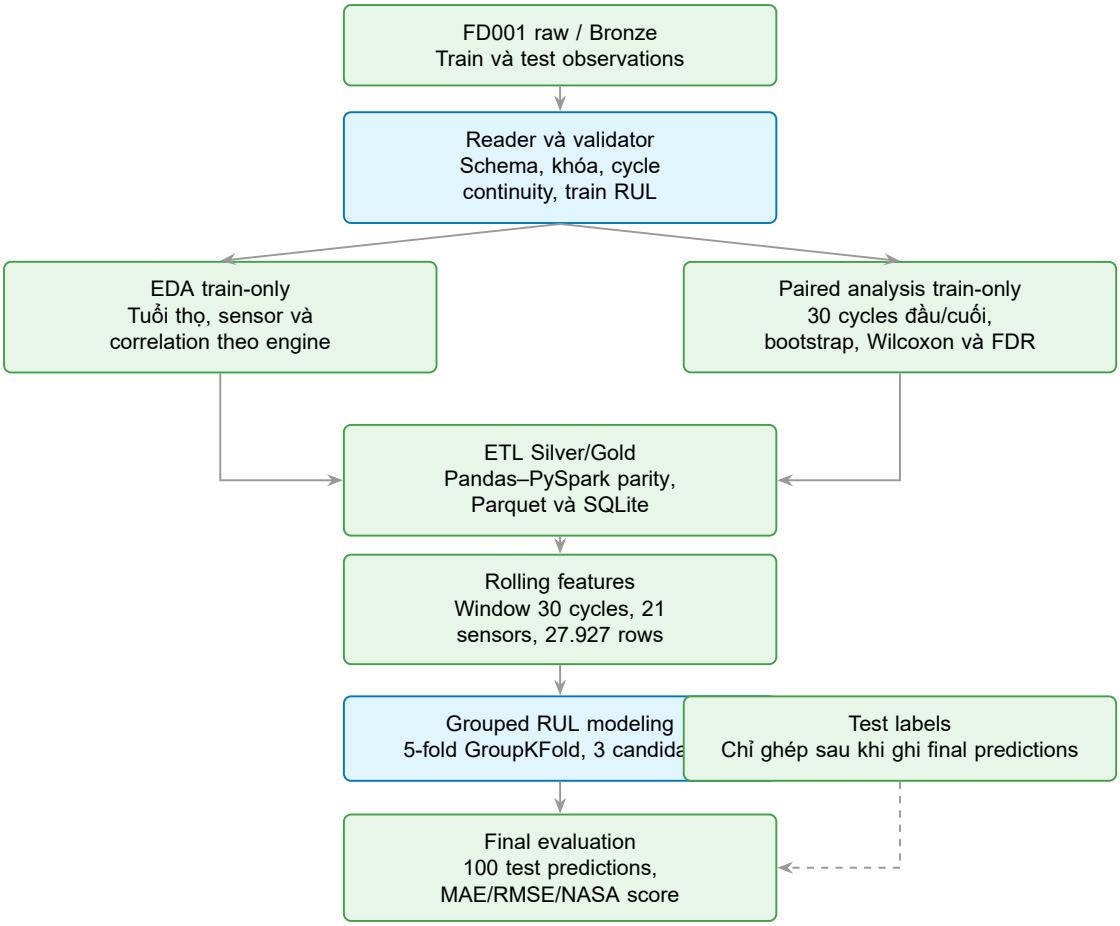
RDD là abstraction nền tảng của Spark cho xử lý phân tán và khả năng phục hồi tính toán [14]. Với dữ liệu có schema, Spark SQL/DataFrame cung cấp các phép xử lý có cấu trúc [1]. Báo cáo dùng Pandas làm reference implementation và PySpark để đối chiếu kết quả ETL, không mặc định rằng Spark nhanh hơn trên FD001.

Bronze giữ dữ liệu nguồn; Silver lưu observations và train targets đã kiểm tra; Gold lưu engine summary và rolling features. Parquet phục vụ lưu bảng phân tích, SQLite phục vụ truy vấn local. Dữ liệu nhân bản chỉ dùng cho benchmark xử lý và không tăng số mẫu khoa học độc lập.

4 Thiết kế phương pháp và lý do lựa chọn

4.1 Luồng xử lý

Hình 1 tóm tắt các bước đã thực hiện: đọc và kiểm tra dữ liệu FD001, phân tích khám phá (EDA), suy luận thống kê theo engine, ETL qua các lớp Silver/Gold, xây dựng rolling features và đánh giá mô hình RUL. Các bước được giữ tách biệt để phân biệt dữ liệu nguồn, dữ liệu đã kiểm tra và dữ liệu suy ra.



Hình 1: Luồng xử lý từ dữ liệu FD001 đến EDA, suy luận thống kê, ETL, rolling features và đánh giá RUL. Nhãn test chỉ được ghép sau khi final predictions đã được ghi.

Dữ liệu raw được giữ bất biến. Reader và validator tạo observations cùng nhãn RUL train trong bộ nhớ. EDA và suy luận thống kê chỉ đọc train. ETL ghi observations và train targets vào Silver Parquet, đồng thời tạo engine summary ở Gold Parquet và SQLite. Rolling features được tạo từ Silver observations và ghi ở Gold. Pipeline modeling join rolling features train với train targets, ghi final predictions cho test trước, rồi mới đọc nhãn test để đánh giá cuối. Cách tách lớp giúp xác định ranh giới của từng phép biến đổi và kiểm tra việc mất hoặc nhân bản dòng.

Pandas là reference implementation; PySpark tái hiện ba boundary và parity checker xác

nhận schema, khóa, số dòng và giá trị số khớp nhau với sai khác lớn nhất bằng 0 trong lần chạy local có JRE. Kết quả benchmark chỉ đo chi phí tạo engine summary trên dữ liệu nhân bản, không tạo thêm engine độc lập cho phân tích khoa học.

4.2 EDA theo đơn vị động cơ

EDA chỉ sử dụng train. Phân phối tuổi thọ được tính mỗi động cơ một lần, trong khi histogram RUL theo observation phản ánh số quan sát khác nhau giữa các động cơ. Hai cách tổng hợp trả lời hai câu hỏi khác nhau và không được diễn giải thay thế nhau.

Correlation giữa sensor và RUL được tính bằng Spearman riêng trong từng động cơ, sau đó tóm tắt phân phối giữa các động cơ. Cách này giữ engine là đơn vị tổng hợp và tránh để chuỗi dài có trọng số quá lớn. Correlation chỉ mang tính mô tả, không chứng minh quan hệ nhân quả hoặc hiệu quả dự đoán ngoài mẫu.

Sensor hằng có correlation không xác định và được giữ dưới dạng NaN. Ba động cơ minh họa được chọn gần các tứ phân vị tuổi thọ, không dựa trên hình dạng sensor; chúng không đại diện cho toàn bộ phân phối.

4.3 Đặc trưng từ trailing window

Biểu diễn đã triển khai là trailing window dài 30 cycles, chỉ chứa hiện tại và quá khứ của cùng một động cơ. Ví dụ, mẫu tại cycle 170 dùng cycles 141–170. Mỗi dòng được định danh bởi `(split, engine_id, cycle)`; engine có dưới 30 cycles bị loại khỏi bước tạo window, không padding hoặc rút ngắn window.

Với mỗi 21 sensor, pipeline tạo sáu đặc trưng: giá trị cuối, mean, sample standard deviation (`ddof=1`), min, max và linear slope theo vị trí 1–30. Vì vậy, mỗi dòng có 126 feature columns và ba cột khóa. Artifact gồm 17.731 train windows và 10.196 test windows. Bước này chưa ghép test labels; các lựa chọn phụ thuộc dữ liệu được giữ ở bước modeling riêng để tránh data leakage.

4.4 Mô hình dự đoán RUL

Modeling dùng 17.731 train windows và 126 feature columns từ artifact M3.2. Tập validation được chia bằng GroupKFold 5-fold theo `engine_id`; mọi window của cùng một engine nằm trong cùng một fold validation. Trong training fold, tổng sample weight của mỗi engine bằng 1 để engine dài không chi phối loss chỉ vì có nhiều window hơn.

Ba ứng viên được so sánh là median baseline, Ridge regression và HistGradientBoostingRegressor. Ridge đặt StandardScaler trong pipeline để scaler chỉ fit trên training fold. Metric chọn mô hình là mean engine-level MAE; RMSE và NASA asymmetric score được ghi kèm. Sau khi chọn mô hình, pipeline refit trên toàn bộ train windows và ghi đúng một dự đoán tại cycle cuối

của mỗi test engine. File nhãn `RUL_FD001.txt` chỉ được đọc trong bước final evaluation sau khi `final_predictions.parquet` đã tồn tại.

4.5 Suy luận thống kê

Phân tích suy luận so sánh mean của 30 cycles đầu và 30 cycles cuối trong cùng engine train. Bảng cặp gồm 2.100 dòng (100 engines $\times$ 21 sensors), còn bảng suy luận gồm một dòng cho mỗi sensor. Protocol giữ đủ 21 sensor, dùng mean paired difference, bootstrap percentile 95%, Wilcoxon permutation và Benjamini–Hochberg FDR. Hình phân phối và effect size được đọc cùng khoảng tin cậy và số dấu của từng sensor; kết quả này mô tả thay đổi trong protocol đầu/cuối, không phải bằng chứng chọn feature hoặc utility dự đoán RUL ngoài mẫu.

5 Protocol suy luận thống kê

5.1 Mục tiêu và phạm vi

Mục tiêu là đánh giá mức thay đổi của sensor giữa đầu và cuối vòng đời trong cùng engine trên tập train FD001. Phân tích dùng trung bình **30 cycles đầu và 30 cycles cuối**, chỉ đọc train và dùng engine làm đơn vị ghép cặp. Các giả thuyết phát sinh từ EDA được diễn giải là khám phá.

Phân tích ghép cặp (paired analysis), chênh lệch cặp trung bình, bootstrap 95%, Wilcoxon khi phù hợp, Benjamini–Hochberg và seed 42 thuộc cấu hình hiện có. Kết quả được tính bằng mean paired difference, percentile bootstrap, Wilcoxon permutation và BH trên họ 21 sensors; bảng và hình chỉ sử dụng train.

5.2 Định nghĩa cặp đầu/cuối và ví dụ

Với $x_{i,t,s}$ là giá trị sensor s của engine i ở cycle t, và T_i là cycle cuối của train engine đó, báo cáo định nghĩa ba đại lượng:

$$E_{i,s} = \frac{1}{30} \sum_{t=1}^{30} x_{i,t,s}. \tag{4}$$

$$L_{i,s} = \frac{1}{30} \sum_{t=T_i-29}^{T_i} x_{i,t,s}. \tag{5}$$

$$D_{i,s} = L_{i,s} - E_{i,s}. \tag{6}$$

Trong đó:

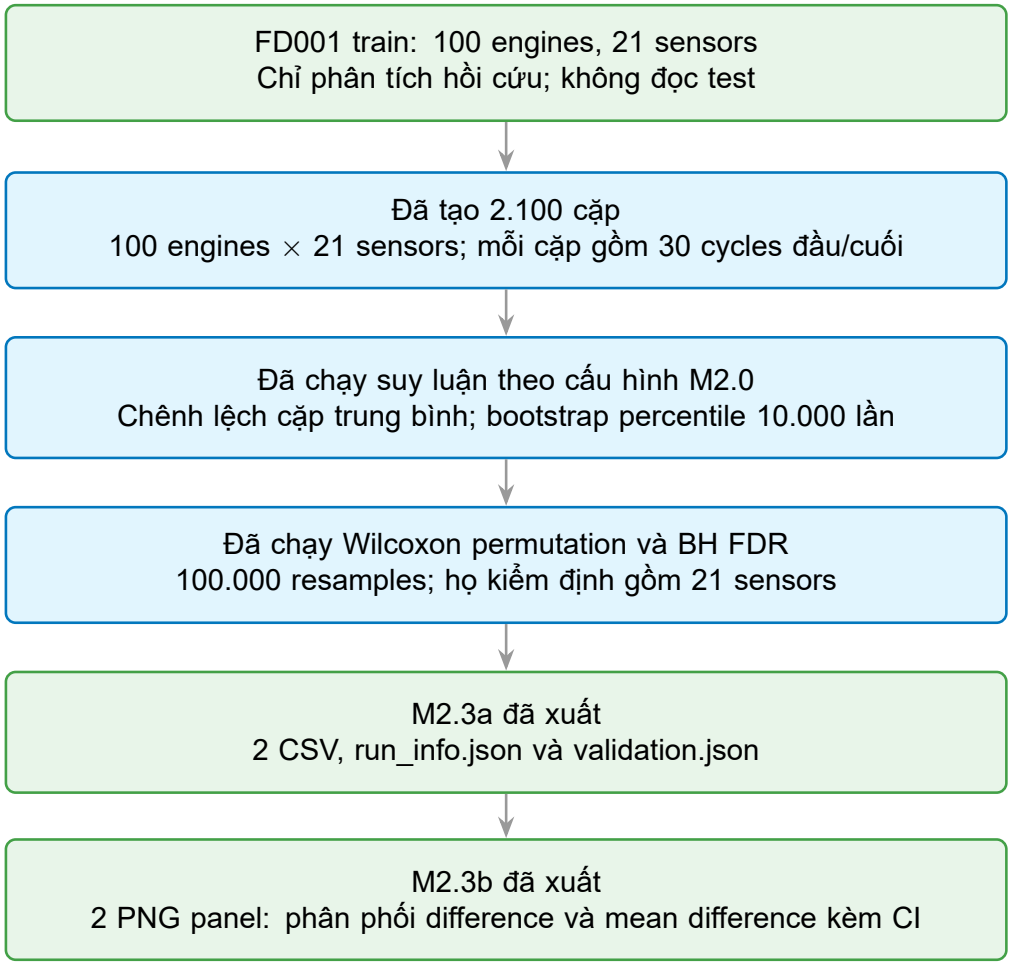
+ $x_{i,t,s}$: giá trị của sensor s trên engine i tại cycle t, giữ đơn vị gốc ẩn danh của sensor đó.
+ T_i: cycle cuối của engine i trong tập train, tính bằng cycle.
+ $E_{i,s}$: mean của sensor s trên 30 cycles đầu của engine i.
+ $L_{i,s}$: mean của sensor s trên 30 cycles cuối của engine i.
+ $D_{i,s}$: chênh lệch giữa mean cuối và mean đầu của sensor s trên engine i, cùng đơn vị với sensor s.

Các Công thức 4–6 là định nghĩa protocol của báo cáo, không phải công thức do bài báo C-MAPSS đề xuất. Chúng được dùng vì mục tiêu ở đây là so sánh cùng một sensor trong cùng engine giữa đầu và cuối vòng đời; lấy mean trên 30 cycles làm giảm phụ thuộc vào một điểm đo đơn lẻ vốn có noise trong mô phỏng C-MAPSS [9]. Nếu $D_{i,s} > 0$, mức trung bình cuối đời lớn hơn đầu đời. Mỗi engine đóng góp một cặp cho mỗi sensor, bất kể tuổi thọ. Hai đoạn phải đủ dữ liệu, liên tục và không chồng lấn. Tuổi thọ train nhỏ nhất là 128 cycles, nên hai đoạn 30 cycles không chồng lấn.

Ví dụ tính chỉ số: với engine có $T_i = 200$, đoạn đầu là cycles 1–30 và đoạn cuối là cycles 171–200. Không nhầm đoạn cuối đời này với window 141–170 dùng dự đoán tại cycle 170. T_i được biết khi phân tích hồi cứu train; không được dùng để xác định đặc trưng ở thời điểm dự

đoán trên test.

Giữ độ dài bằng nhau giúp so sánh hai mức tín hiệu trên cùng số cycles. Tuy nhiên, 30 cycles chiếm tỷ lệ vòng đời khác nhau giữa các engine. Protocol này trả lời câu hỏi về hai đoạn có độ dài tuyệt đối cố định, không phải so sánh cùng phần trăm tuổi thọ và cũng không tự xác định chính xác thời điểm bắt đầu suy giảm.



Hình 2: Thiết kế phân tích đầu/cuối vòng đời theo engine trên tập train FD001.

5.3 Đại lượng cần báo cáo và điều kiện diễn giải

Bảng kết quả M2.3a giữ đủ 21 sensors, không chọn danh sách theo thứ hạng correlation của EDA. Ba operational settings không được gộp vào cùng họ giả thuyết sensor. Bảng ghi số cặp hợp lệ, số chênh lệch bằng 0, chênh lệch cặp trung bình, khoảng tin cậy, thống kê kiểm định, p-value gốc, p-value hiệu chỉnh và trạng thái có thể kiểm định.

Dùng chênh lệch cặp trung bình làm đại lượng chính theo đơn vị gốc:

$$\widehat{\Delta}_s = \frac{1}{m}\sum_{i=1}^{m} D_{i,s}. \tag{7}$$

Trong đó:

+ s: sensor đang được phân tích.
+ m: số engine có cặp đầu/cuối hợp lệ.
+ $D_{i,s}$: chênh lệch của sensor s trên engine i, cùng đơn vị với sensor s.
+ $\widehat{\Delta}_s$: mean paired difference của sensor s, cùng đơn vị với sensor s.

Mean paired difference trong Công thức 7 được dùng vì nó trả lời trực tiếp câu hỏi "cuối đời thay đổi trung bình bao nhiêu so với đầu đời" theo đơn vị gốc. Đây là đại lượng mô tả của báo cáo, không phải một score chuẩn hóa được đề xuất riêng cho C-MAPSS. Vì các sensor có thang đo khác nhau, không so sánh trực tiếp độ lớn của $\widehat{\Delta}_s$ giữa hai sensor. Độ bất định được ước lượng bằng bootstrap: lấy mẫu lại các engine cùng cặp E, L để tạo khoảng tin cậy 95% cho $\widehat{\Delta}_s$ theo phương pháp bootstrap của Efron và hàm SciPy đã khóa phiên bản [3, 10]. Cấu hình dùng 10.000 resamples và percentile interval.

Wilcoxon hai phía kiểm định giả thuyết trung vị của chênh lệch bằng 0. Việc diễn giải dựa vào giả định phân phối chênh lệch đối xứng quanh trung vị và các cặp độc lập giữa engine [12]. Cần xem phân phối chênh lệch và ghi rõ giới hạn của giả định đối xứng; bootstrap của trung bình không thay thế giả định này. Sensor hằng hoặc có toàn bộ $D_{i,s} = 0$ vẫn phải xuất hiện trong bảng dưới trạng thái không có biến thiên quan sát được. Không biến p-value không xác định thành 0 hoặc tự kết luận sensor có giá trị dự đoán bằng 0.

Benjamini–Hochberg áp dụng cho một họ kiểm định đã xác định trước [2, 11]. Cấu hình M2.0 giữ họ gồm 21 sensors, dùng giá trị 1 chỉ trong vector hiệu chỉnh cho trường hợp không kiểm định được, đồng thời giữ p-value gốc là không áp dụng và ghi rõ trạng thái trong bảng. Quy ước này tránh thu nhỏ họ giả thuyết theo kết quả thuận lợi; nó không sửa được p-value thiếu hợp lệ hoặc bảo đảm FDR dưới mọi dạng phụ thuộc giữa sensors. Các khoảng tin cậy bootstrap riêng lẻ không phải simultaneous intervals.

5.4 Cấu hình suy luận

Các giá trị sau là cấu hình sử dụng khi tính kết quả; không lấy default của thư viện làm quyết định ngầm.

- Đại lượng chính: mean paired difference; kèm median, số dấu dương, số dấu âm và số zero để xem phân phối lệch.
- Bootstrap: 10.000 lần lấy mẫu lại 100 engines có hoàn lại, seed 42; dùng percentile interval tại các quantile 2,5% và 97,5%. Ghi rõ giới hạn độ bao phủ hữu hạn; không mặc định tương đương BCa. Trường hợp phân phối bootstrap suy biến phải gắn nhãn, không diễn giải khoảng bằng 0 là chắc chắn tuyệt đối.
- Wilcoxon: hai phía, zero method `wilcox`, không dùng continuity correction và không dùng `auto`. Dùng permutation với 100.000 resamples và RNG cố định cho sensors có thể kiểm định. Zero được xác định bằng `difference == 0.0` sau tính mean `float64`; không tự làm tròn bằng một ngưỡng tùy ý cho mọi sensor.

- All-zero và p-value thiếu: nếu toàn bộ $D_{i,s} = 0$, giữ sensor trong bảng với trạng thái all-zero, `raw_p=NA` và không diễn giải như một kiểm định thành công. Lỗi số bất ngờ phải làm lần chạy dừng thay vì biến thành p-value hợp lệ.
- FDR: dùng Benjamini–Hochberg với $q = 0{,}05$ trên đúng họ 21 sensors bằng hàm SciPy đã cố định trong metadata. Với p-value không áp dụng, chỉ dùng sentinel 1,0 bên trong vector hiệu chỉnh; output vẫn giữ `raw_p=NA` và trạng thái riêng.
- RNG: dùng seed 42 và tách luồng theo sensor và mục đích bootstrap/permutation để đổi thứ tự chạy không đổi kết quả của cùng sensor.

Cấu hình này được áp dụng nhất quán cho toàn bộ 21 sensor. Kiểm tra gồm chỉ số 30 cycles đúng trong từng engine, không chồng đoạn, ghép cặp một–một và trường hợp hằng/ties/zeros. Bootstrap lấy mẫu theo engine và giữ cặp; chạy lại với cùng input và seed phải tái lập kết quả.

Kết luận thống kê chỉ áp dụng cho thay đổi trên train mô phỏng theo protocol này. Phát hiện thay đổi không chứng minh quan hệ nhân quả hoặc hiệu quả dự đoán ngoài mẫu. Lựa chọn sensor cho mô hình vẫn phải học trong training fold; không dùng danh sách có ý nghĩa thống kê trên toàn train để chọn feature trước cross-validation.

6 Kết quả và diễn giải

6.1 Kiểm tra dữ liệu

Validation đạt cho observations train, nhãn RUL train và observations test. Tập train có 20.631 dòng, tập test có 13.096 dòng; mỗi tập gồm 100 engine. Một trăm test labels hợp lệ về cấu trúc nhưng được giữ tách khỏi EDA và suy luận trên train. Các kiểm tra schema, khóa duy nhất, kiểu số hữu hạn và cycle liên tục đều đạt.

6.2 ETL Silver/Gold và benchmark

Pipeline Pandas và PySpark đọc dữ liệu sau khi kiểm tra data contract. Silver gồm 33.727 observations train/test và 20.631 train targets. Gold gồm 200 dòng engine summary trong Parquet và bảng truy vấn tương ứng trong SQLite. Các bảng được đọc lại sau khi ghi.

Bảng 7: Benchmark tạo engine summary trên observations nhân bản trong bộ nhớ; dữ liệu nhân bản không dùng để huấn luyện.

Scale	Input rows	Engine rows	Cold (s)	Warm (s)
1×	33.727	200	0,0195	0,0196
10×	337.270	2.000	0,1589	0,1474
100×	3.372.700	20.000	1,6752	1,6732

Benchmark đo chi phí của phép tổng hợp cục bộ, không phải bằng chứng về xử lý phân tán. Pandas và PySpark khớp schema, khóa, số dòng và giá trị số cho ba bảng Silver/Gold; sai khác tuyệt đối lớn nhất đều bằng 0,0.

6.3 Rolling features

Rolling features dài 30 cycles gồm 17.731 train windows và 10.196 test windows, tổng cộng 27.927 dòng. Mỗi dòng có khóa `(split, engine_id, cycle)` và 126 feature columns cho 21 sensor, gồm `last`, `mean`, sample `std`, `min`, `max` và `slope`. Test labels không được ghép trong bước này.

Bảng 8: Số rolling windows theo split.

Split	Engines	Rolling windows
Train	100	17.731
Test	100	10.196
Tổng	200	27.927

Validator kiểm tra schema, khóa duy nhất, thứ tự canonical, số hữu hạn và đủ trailing window

trong từng engine. Tests tính tay xác nhận sample standard deviation, slope, không dùng tương lai và không nối hai engine.

6.4 Mô hình RUL và đánh giá cuối

Pipeline modeling so sánh ba ứng viên trên 5 folds theo engine. Bảng 9 trình bày metric trung bình qua folds; mô hình được chọn là HistGradientBoostingRegressor vì có mean engine-level MAE thấp nhất.

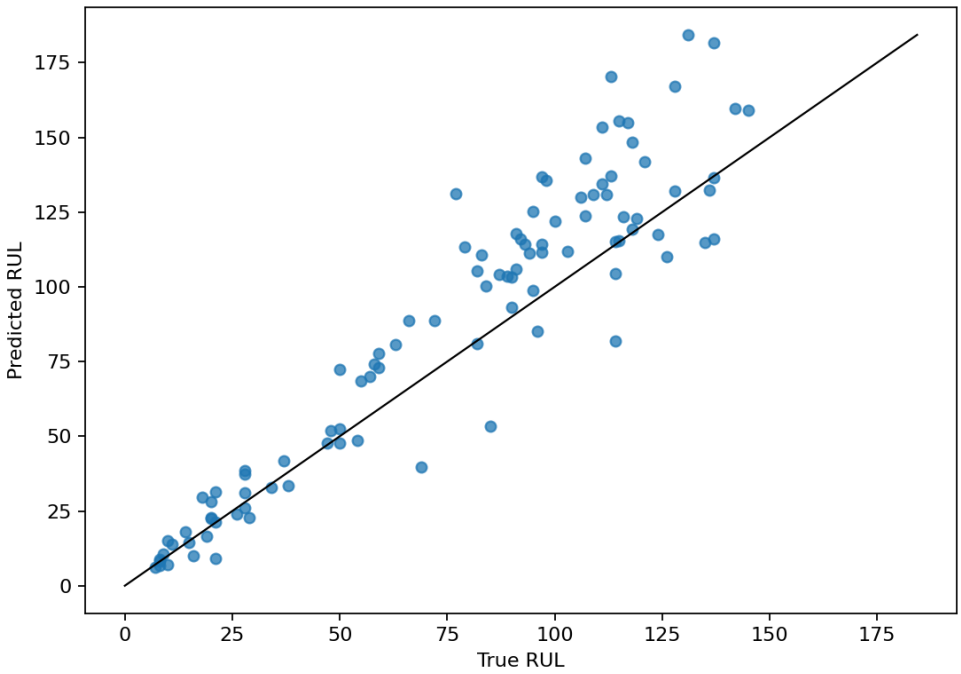
Bảng 9: Kết quả grouped cross-validation trên 17.731 train windows. NASA score ở bảng này là tổng theo validation windows, chỉ dùng để so sánh trong cùng protocol.

Model	Mean engine MAE	RMSE trung bình	Chọn
`median_baseline`	46,9543	62,9006	Không
`ridge_alpha_1`	23,9922	35,9467	Không
`hist_gradient_boosting`	20,2143	33,5479	Có

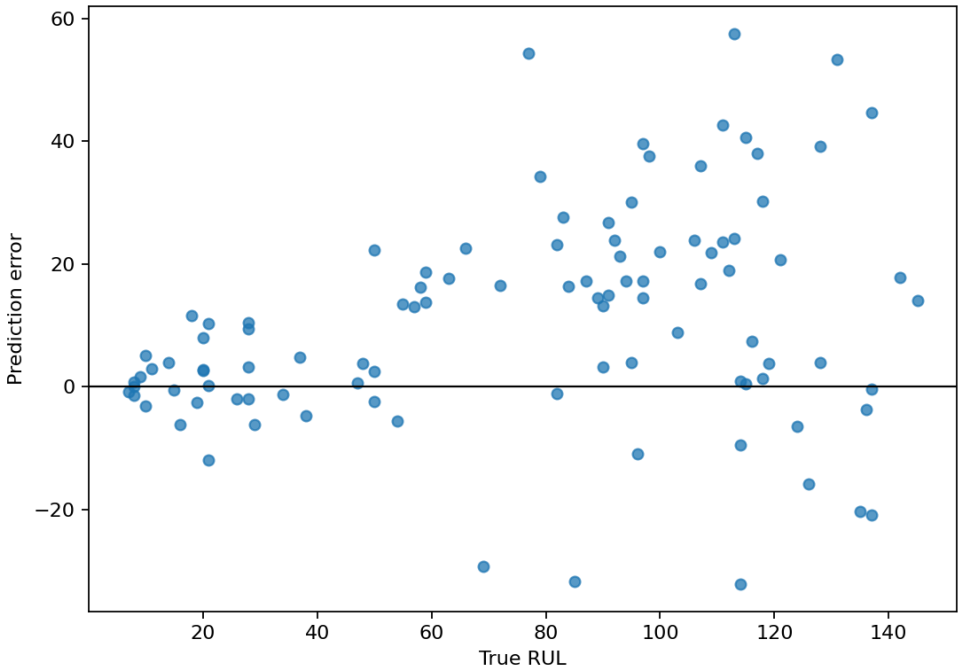
Sau khi refit trên toàn bộ train windows, pipeline ghi 100 final predictions ở cycle cuối của 100 test engines rồi mới ghép nhãn NASA. Kết quả cuối được tóm tắt trong Bảng 10.

Bảng 10: Đánh giá model được chọn trên 100 test engines; khoảng tin cậy bootstrap lấy mẫu lại engine 10.000 lần, seed 42.

Metric	Giá trị	Bootstrap CI 95%
Mean engine MAE	15,3490	[13,2594; 17,3941]
RMSE	20,5413	[17,3442; 23,5850]
NASA score	1.513,2577	[743,4999; 2.476,2743]



Hình 3: RUL thật và RUL dự đoán của 100 test engines. Đường chéo biểu diễn dự đoán hoàn hảo.



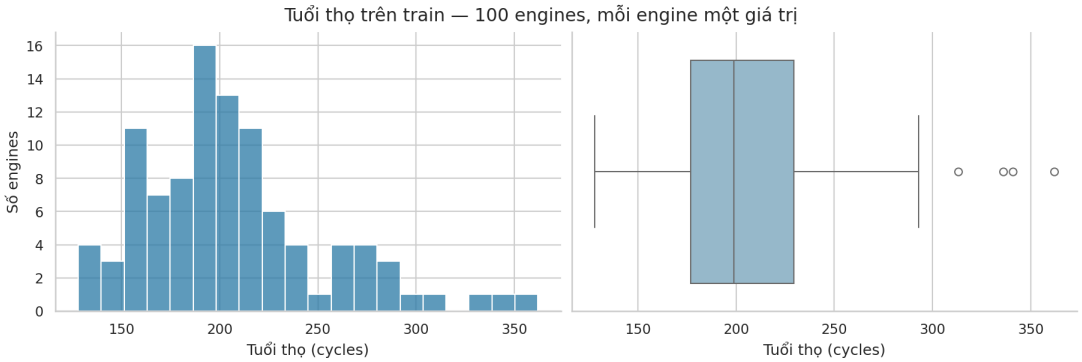
Hình 4: Sai số `prediction - true` theo RUL thật của 100 test engines.

Sai số tuyệt đối trung bình trên nhóm RUL thật 0–30 cycles là 4,3830, thấp hơn các nhóm 31–60, 61–90 và trên 90 cycles. Mười sai số lớn nhất đều là dự đoán cao hơn nhãn thật từ khoảng 37,6 đến 57,5 cycles, nên NASA score bị chi phối bởi những trường hợp dự đoán quá lạc quan. Kết quả này là đánh giá trên FD001 mô phỏng, không phải bảo đảm khái quát hóa sang fault mode hoặc điều kiện vận hành khác.

6.5 Tuổi thọ động cơ trên train

Bảng 11: Thống kê tuổi thọ, đơn vị cycle.

Thống kê	Giá trị	Thống kê	Giá trị
Trung bình	206,31	Độ lệch chuẩn mẫu	46,34
Nhỏ nhất	128	Lớn nhất	362
Tứ phân vị 25%	177	Trung vị	199
Tứ phân vị 75%	229,25	Số động cơ	100



Hình 5: Phân phối tuổi thọ train; mỗi động cơ đóng góp một giá trị.

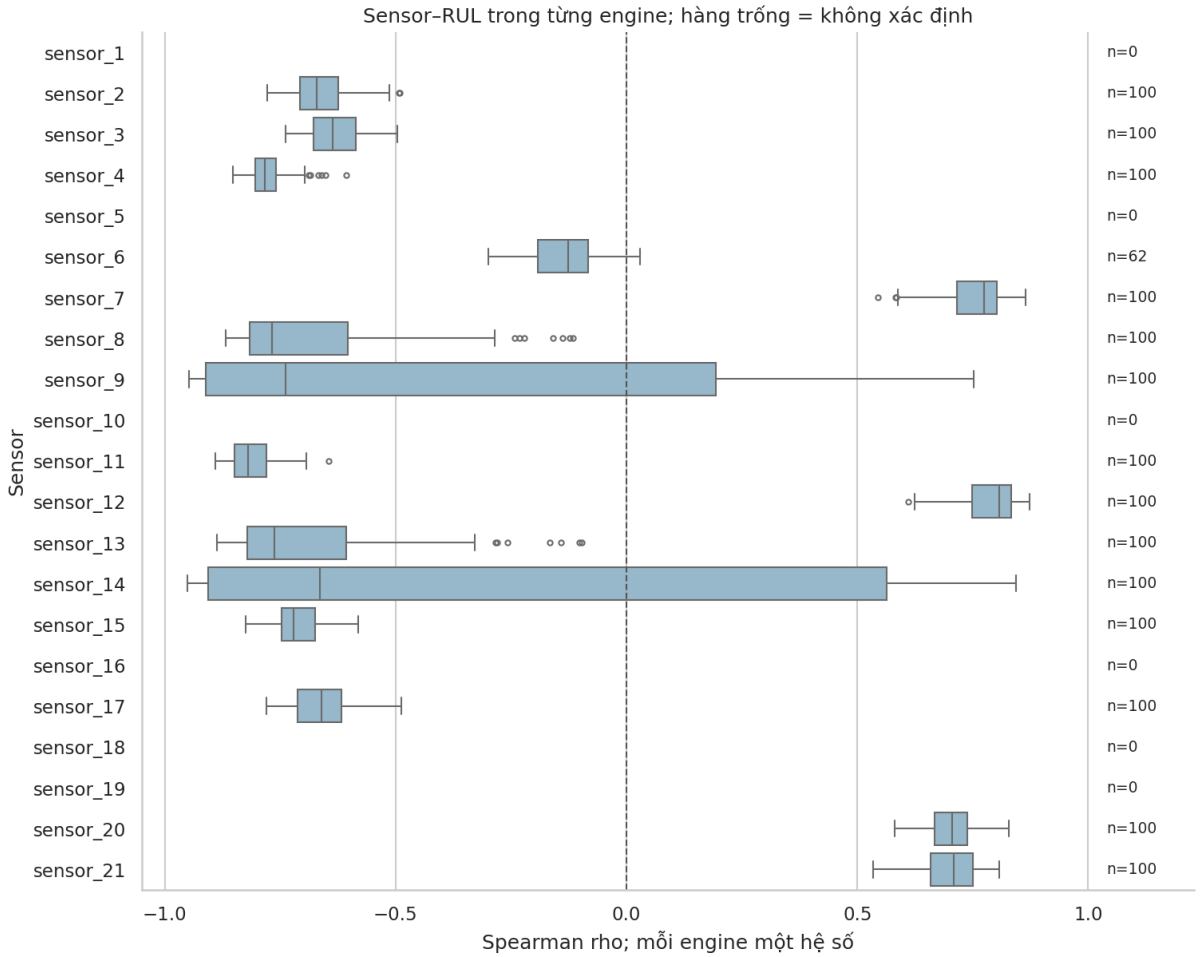
Tuổi thọ dao động từ 128 đến 362 cycles; trung bình lớn hơn trung vị. Vì độ dài chuỗi khác nhau, báo cáo phân biệt rõ tổng hợp theo observation và theo engine. Các con số không được diễn giải thành tuổi thọ theo giờ hoặc ngày.

6.6 Sensor hằng và correlation theo engine

Bảng 12: Một số dòng minh họa; bảng này không dùng để chọn sensor.

Sensor	Động cơ xác định	Tương quan trung vị	Không xác định
1	0	Không xác định	100
6	62	−0,1264	38
11	100	−0,8190	0
12	100	0,8075	0

Sensor 1, 5, 10, 16, 18 và 19 hằng trên train. Sensor 6 hằng trong 38/100 engine; correlation chỉ xác định trong 62 engine còn lại. Sensor 11 có tương quan trung vị âm, còn sensor 12 có tương quan trung vị dương với RUL. Đây là liên hệ mô tả trên train, không phải bằng chứng nhân quả hoặc hiệu quả ngoài mẫu.



Hình 6: Phân phối correlation Spearman giữa các engine train. Sensor hằng được giữ là không xác định.

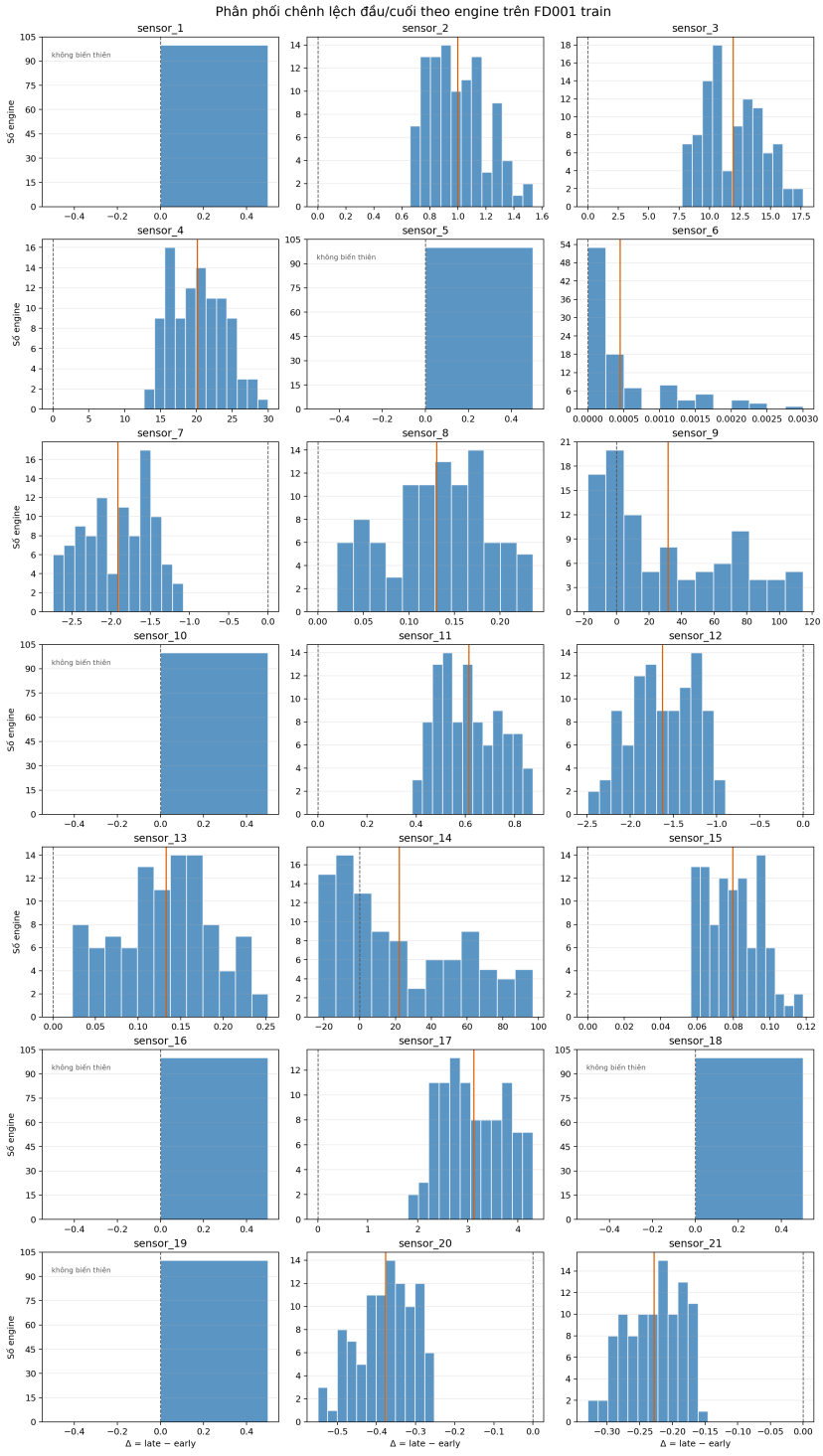
6.7 Suy luận đầu/cuối vòng đời trên train

Bảng cặp gồm 2.100 dòng, tương ứng 100 engine train nhân 21 sensor. Mỗi dòng dùng mean của 30 cycles đầu, mean của 30 cycles cuối và chênh lệch cuối trừ đầu. Bảng suy luận giữ đủ 21 sensor.

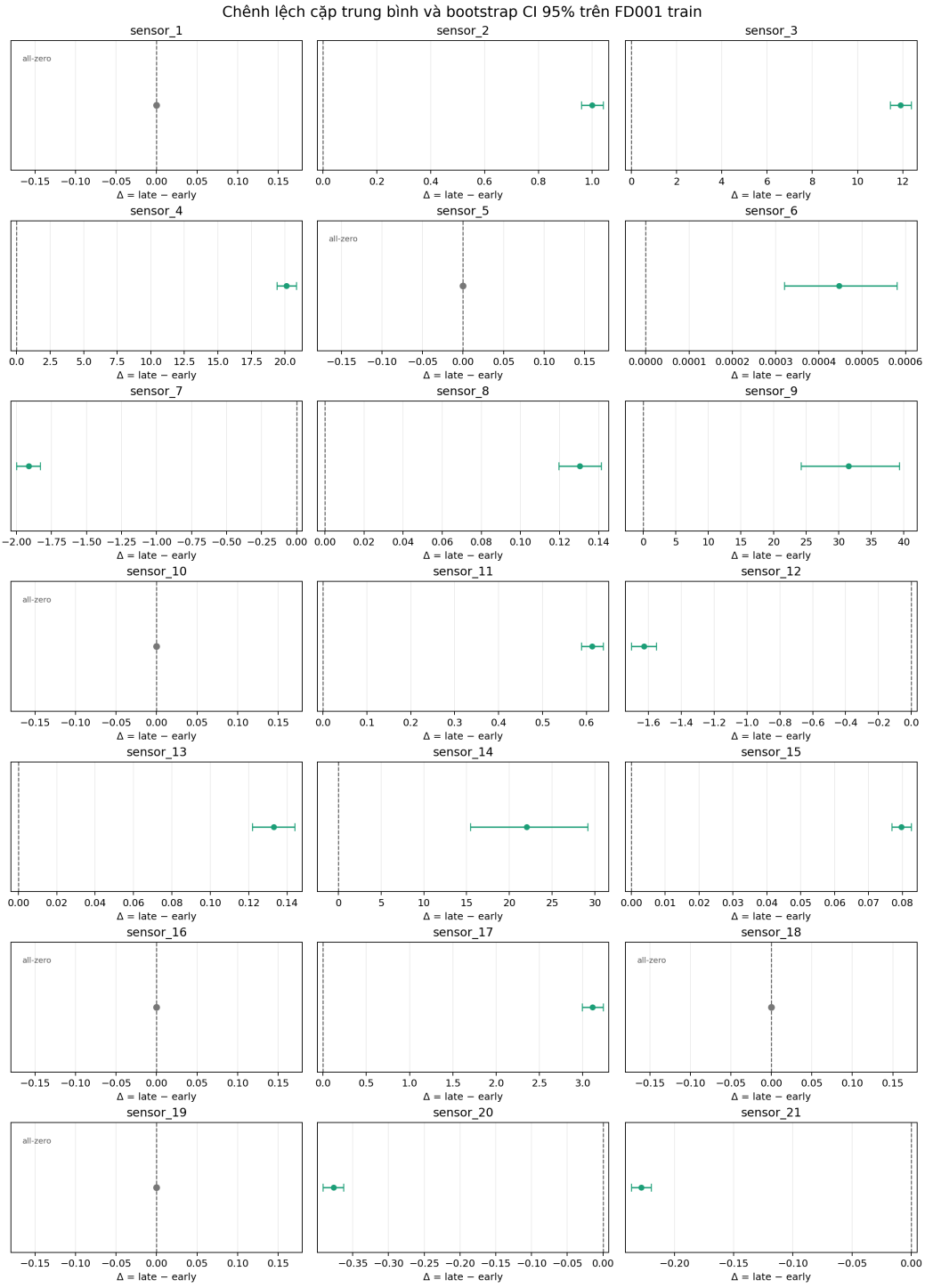
Trong 21 sensor, sáu sensor có toàn bộ chênh lệch bằng 0: `sensor_1`, `sensor_5`, `sensor_10`, `sensor_16`, `sensor_18` và `sensor_19`. Mười lăm sensor còn lại kiểm định được; với cấu hình Wilcoxon permutation và Benjamini–Hochberg trên họ 21 sensor, các p-value đã hiệu chỉnh đều xấp xỉ 0,000028.

Bảng 13: Một số kết quả tiêu biểu. Giá trị là chênh lệch cuối trừ đầu theo đơn vị gốc của từng sensor.

Sensor	Chênh lệch trung bình	Khoảng tin cậy bootstrap 95%	Dấu chính
`sensor_4`	20,128097	[19,402735; 20,865606]	Tăng
`sensor_7`	-1,913527	[-1,999307; -1,829684]	Giảm
`sensor_11`	0,613257	[0,589017; 0,637944]	Tăng
`sensor_12`	-1,627190	[-1,704288; -1,552119]	Giảm
`sensor_20`	-0,377007	[-0,391534; -0,363173]	Giảm



Hình 7: Phân phối chênh lệch cuối trừ đầu của 100 engine cho từng sensor.



Hình 8: Mean paired difference và bootstrap CI 95% theo từng sensor.

Kết quả cho thấy nhiều sensor thay đổi có hệ thống giữa hai đoạn 30 cycles trên train FD001 theo protocol đã chọn. Sensor 4 có chênh lệch trung bình khoảng 20,13, còn sensor 7 giảm khoảng 1,91; hai giá trị không được xếp hạng trực tiếp vì khác thang đo. Sensor 6 có 47 chênh lệch dương và 53 chênh lệch bằng 0 nhưng mean difference chỉ khoảng 0,00045; p-value nhỏ không đồng nghĩa thay đổi lớn. Phân tích này không phải bằng chứng chọn feature hoặc utility dự đoán RUL ngoài mẫu.

7 Khó khăn, bài học và giới hạn

Khó khăn đầu tiên là phân biệt dữ liệu đủ để tính nhãn với dữ liệu đủ để dự đoán trong thực tế. Train cho phép biết T_i sau khi chuỗi kết thúc, còn tại cycle t thì T_i chưa biết. Vì vậy, việc tạo nhãn được tách khỏi feature và không dùng tuổi thọ đầy đủ để tạo biến đầu vào.

Khó khăn thứ hai là xác định đơn vị phân tích. Một engine sống lâu tạo nhiều dòng hơn, trong khi các quan sát liền nhau có dependence theo thời gian. Tăng số dòng không đồng nghĩa tăng số mẫu độc lập; EDA và bootstrap vì vậy giữ engine là đơn vị tổng hợp hoặc lấy mẫu.

Khó khăn thứ ba là giới hạn của dữ liệu mô phỏng và benchmark cục bộ. PySpark có thể chịu overhead trên FD001 nhỏ; parity xác nhận tính tương đương nhưng không cho phép suy ra công cụ phân tán luôn nhanh hơn Pandas.

Các kết quả trong báo cáo đã có validation dữ liệu, EDA, ETL, rolling features, suy luận thay đổi sensor đầu/cuối và đánh giá mô hình RUL đầu tiên. Tuy nhiên, đây vẫn là kết quả trên FD001 mô phỏng với một điều kiện vận hành và một dạng lỗi. Chưa có forecast ARIMA hoặc dashboard trong tập bằng chứng này; vì vậy không kết luận lịch bảo trì nào có lợi hoặc mô hình sẽ hoạt động tương tự trên dữ liệu thực tế. Đây là giới hạn phạm vi kết quả, không phải kết luận rằng các phương pháp đó thất bại.

Tài liệu tham khảo

[1] Apache Software Foundation. Spark SQL and DataFrames. Apache Spark documentation. Accessed 2026-09-22; implementation version not selected. URL: https://spark.apache.org/docs/latest/sql-programming-guide.html.

[2] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi:10.1111/j.2517-6161.1995.tb02031.x.

[3] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi:10.1214/aos/1176344552.

[4] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3 edition, 2021. Online edition, sections 5.4, 9.1, 9.5 and 9.7; accessed 2026-09-22. URL: https://otexts.com/fpp3/.

[5] Xiang Li, Qian Ding, and Jian-Qiao Sun. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172:1–11, 2018. doi:10.1016/j.ress.2017.11.021.

[6] NASA. CMAPSS jet engine simulated data. NASA Open Data Portal. Accessed 2026-09-20; project uses FD001. URL: https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data.

[7] Emmanuel Ramasso and Abhinav Saxena. Review and analysis of algorithmic approaches developed for prognostics on CMAPSS dataset. In *Annual Conference of the PHM Society*, volume 6, 2014. Issue 1. URL: https://papers.phmsociety.org/index.php/phmconf/article/view/2512, doi:10.36001/phmconf.2014.v6i1.2512.

[8] Abhinav Saxena, Jose Celaya, Bhaskar Saha, Sankalita Saha, and Kai Goebel. Metrics for offline evaluation of prognostic performance. *International Journal of Prognostics and Health Management*, 1(1), 2010. URL: https://papers.phmsociety.org/index.php/ijphm/article/view/1336, doi:10.36001/ijphm.2010.v1i1.1336.

[9] Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, pages 1–9. IEEE, 2008. doi:10.1109/PHM.2008.4711414.

[10] SciPy developers. scipy.stats.bootstrap. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.bootstrap.html.

[11] SciPy developers. scipy.stats.false_discovery_control. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.false_discovery_control.html.

[12] SciPy developers. scipy.stats.wilcoxon. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.wilcoxon.html.

[13] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi:10.2307/3001968.

[14] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 15–28. USENIX Association, 2012. URL: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia.